Figure 7.16 does the median-of-three partitioning with all the side effects described. It may seem that it is only slightly inefficient to compute the pivot by a method that does not actually sort `a[left]`, `a[center]`, and `a[right]`, but, surprisingly, this produces bad results (see Exercise 7.51).

The real heart of the quicksort routine is in Figure 7.17. It includes the partitioning and recursive calls. There are several things worth noting in this implementation. Line 16 initializes `i` and `j` to 1 past their correct values, so that there are no special cases to consider. This initialization depends on the fact that median-of-three partitioning has

```
 1  /**
 2   * Internal quicksort method that makes recursive calls.
 3   * Uses median-of-three partitioning and a cutoff of 10.
 4   * a is an array of Comparable items.
 5   * left is the left-most index of the subarray.
 6   * right is the right-most index of the subarray.
 7   */
 8  template <typename Comparable>
 9  void quicksort( vector<Comparable> & a, int left, int right )
10  {
11      if( left + 10 <= right )
12      {
13          const Comparable & pivot = median3( a, left, right );
14
15              // Begin partitioning
16          int i = left, j = right - 1;
17          for( ; ; )
18          {
19              while( a[ ++i ] < pivot ) { }
20              while( pivot < a[ --j ] ) { }
21              if( i < j )
22                  std::swap( a[ i ], a[ j ] );
23              else
24                  break;
25          }
26
27          std::swap( a[ i ], a[ right - 1 ] );  // Restore pivot
28
29          quicksort( a, left, i - 1 );     // Sort small elements
30          quicksort( a, i + 1, right );    // Sort large elements
31      }
32      else  // Do an insertion sort on the subarray
33          insertionSort( a, left, right );
34  }
```

Figure 7.17 Main quicksort routine

```
16          int i = left + 1, j = right - 2;
17          for( ; ; )
18          {
19              while( a[ i ] < pivot ) i++;
20              while( pivot < a[ j ] ) j--;
21              if( i < j )
22                  std::swap( a[ i ], a[ j ] );
23              else
24                  break;
25          }
```

Figure 7.18 A small change to quicksort, which breaks the algorithm

some side effects; this program will not work if you try to use it without change with a simple pivoting strategy, because `i` and `j` start in the wrong place and there is no longer a sentinel for `j`.

The swapping action at line 22 is sometimes written explicitly, for speed purposes. For the algorithm to be fast, it is necessary to force the compiler to compile this code inline. Many compilers will do this automatically if `swap` is declared using `inline`, but for those that do not, the difference can be significant.

Finally, lines 19 and 20 show why quicksort is so fast. The inner loop of the algorithm consists of an increment/decrement (by 1, which is fast), a test, and a jump. There is no extra juggling as there is in mergesort. This code is still surprisingly tricky. It is tempting to replace lines 16 to 25 with the statements in Figure 7.18. This does not work, because there would be an infinite loop if `a[i] = a[j] = pivot`.

7.7.5 Analysis of Quicksort

Like mergesort, quicksort is recursive; therefore, its analysis requires solving a recurrence formula. We will do the analysis for a quicksort, assuming a random pivot (no median-of-three partitioning) and no cutoff for small arrays. We will take $T(0) = T(1) = 1$, as in mergesort. The running time of quicksort is equal to the running time of the two recursive calls plus the linear time spent in the partition (the pivot selection takes only constant time). This gives the basic quicksort relation

$$T(N) = T(i) + T(N - i - 1) + cN \tag{7.1}$$

where $i = |S_1|$ is the number of elements in S_1. We will look at three cases.

Worst-Case Analysis

The pivot is the smallest element, all the time. Then $i = 0$, and if we ignore $T(0) = 1$, which is insignificant, the recurrence is

$$T(N) = T(N - 1) + cN, \quad N > 1 \tag{7.2}$$

We telescope, using Equation (7.2) repeatedly. Thus,

$$T(N-1) = T(N-2) + c(N-1) \tag{7.3}$$

$$T(N-2) = T(N-3) + c(N-2) \tag{7.4}$$

$$\vdots$$

$$T(2) = T(1) + c(2) \tag{7.5}$$

Adding up all these equations yields

$$T(N) = T(1) + c\sum_{i=2}^{N} i = \Theta(N^2) \tag{7.6}$$

as claimed earlier. To see that this is the worst possible case, note that the total cost of all the partitions in recursive calls at depth d must be at most N. Since the recursion depth is at most N, this gives an $O(N^2)$ worst-case bound for quicksort.

Best-Case Analysis

In the best case, the pivot is in the middle. To simplify the math, we assume that the two subarrays are each exactly half the size of the original, and although this gives a slight overestimate, this is acceptable because we are only interested in a Big-Oh answer.

$$T(N) = 2T(N/2) + cN \tag{7.7}$$

Divide both sides of Equation (7.7) by N.

$$\frac{T(N)}{N} = \frac{T(N/2)}{N/2} + c \tag{7.8}$$

We will telescope using this equation:

$$\frac{T(N/2)}{N/2} = \frac{T(N/4)}{N/4} + c \tag{7.9}$$

$$\frac{T(N/4)}{N/4} = \frac{T(N/8)}{N/8} + c \tag{7.10}$$

$$\vdots$$

$$\frac{T(2)}{2} = \frac{T(1)}{1} + c \tag{7.11}$$

We add all the equations from (7.8) to (7.11) and note that there are $\log N$ of them:

$$\frac{T(N)}{N} = \frac{T(1)}{1} + c\log N \tag{7.12}$$

which yields

$$T(N) = cN\log N + N = \Theta(N\log N) \tag{7.13}$$

Notice that this is the exact same analysis as mergesort; hence, we get the same answer. That this is the best case is implied by results in Section 7.8.

Average-Case Analysis

This is the most difficult part. For the average case, we assume that each of the sizes for S_1 is equally likely, and hence has probability $1/N$. This assumption is actually valid for our pivoting and partitioning strategy, but it is not valid for some others. Partitioning strategies that do not preserve the randomness of the subarrays cannot use this analysis. Interestingly, these strategies seem to result in programs that take longer to run in practice.

With this assumption, the average value of $T(i)$, and hence $T(N - i - 1)$, is $(1/N)\sum_{j=0}^{N-1} T(j)$. Equation (7.1) then becomes

$$T(N) = \frac{2}{N}\left[\sum_{j=0}^{N-1} T(j)\right] + cN \tag{7.14}$$

If Equation (7.14) is multiplied by N, it becomes

$$NT(N) = 2\left[\sum_{j=0}^{N-1} T(j)\right] + cN^2 \tag{7.15}$$

We need to remove the summation sign to simplify matters. We note that we can telescope with one more equation:

$$(N-1)T(N-1) = 2\left[\sum_{j=0}^{N-2} T(j)\right] + c(N-1)^2 \tag{7.16}$$

If we subtract Equation (7.16) from Equation (7.15), we obtain

$$NT(N) - (N-1)T(N-1) = 2T(N-1) + 2cN - c \tag{7.17}$$

We rearrange terms and drop the insignificant $-c$ on the right, obtaining

$$NT(N) = (N+1)T(N-1) + 2cN \tag{7.18}$$

We now have a formula for $T(N)$ in terms of $T(N-1)$ only. Again the idea is to telescope, but Equation (7.18) is in the wrong form. Divide Equation (7.18) by $N(N+1)$:

$$\frac{T(N)}{N+1} = \frac{T(N-1)}{N} + \frac{2c}{N+1} \tag{7.19}$$

Now we can telescope.

$$\frac{T(N-1)}{N} = \frac{T(N-2)}{N-1} + \frac{2c}{N} \tag{7.20}$$

$$\frac{T(N-2)}{N-1} = \frac{T(N-3)}{N-2} + \frac{2c}{N-1} \tag{7.21}$$

$$\vdots$$

$$\frac{T(2)}{3} = \frac{T(1)}{2} + \frac{2c}{3} \tag{7.22}$$

Adding Equations (7.19) through (7.22) yields

$$\frac{T(N)}{N+1} = \frac{T(1)}{2} + 2c \sum_{i=3}^{N+1} \frac{1}{i} \tag{7.23}$$

The sum is about $\log_e(N+1) + \gamma - \frac{3}{2}$, where $\gamma \approx 0.577$ is known as Euler's constant, so

$$\frac{T(N)}{N+1} = O(\log N) \tag{7.24}$$

And so

$$T(N) = O(N \log N) \tag{7.25}$$

Although this analysis seems complicated, it really is not—the steps are natural once you have seen some recurrence relations. The analysis can actually be taken further. The highly optimized version that was described above has also been analyzed, and this result gets extremely difficult, involving complicated recurrences and advanced mathematics. The effect of equal elements has also been analyzed in detail, and it turns out that the code presented does the right thing.

7.7.6 A Linear-Expected-Time Algorithm for Selection

Quicksort can be modified to solve the *selection problem,* which we have seen in Chapters 1 and 6. Recall that by using a priority queue, we can find the kth largest (or smallest) element in $O(N + k \log N)$. For the special case of finding the median, this gives an $O(N \log N)$ algorithm.

Since we can sort the array in $O(N \log N)$ time, one might expect to obtain a better time bound for selection. The algorithm we present to find the kth smallest element in a set S is almost identical to quicksort. In fact, the first three steps are the same. We will call this algorithm **quickselect.** Let $|S_i|$ denote the number of elements in S_i. The steps of quickselect are

1. If $|S| = 1$, then $k = 1$ and return the element in S as the answer. If a cutoff for small arrays is being used and $|S| \leq$ `CUTOFF`, then sort S and return the kth smallest element.
2. Pick a pivot element, $v \in S$.
3. Partition $S - \{v\}$ into S_1 and S_2, as was done with quicksort.
4. If $k \leq |S_1|$, then the kth smallest element must be in S_1. In this case, return quickselect(S_1, k). If $k = 1 + |S_1|$, then the pivot is the kth smallest element and we can return it as the answer. Otherwise, the kth smallest element lies in S_2, and it is the $(k - |S_1| - 1)$st smallest element in S_2. We make a recursive call and return quickselect$(S_2, k - |S_1| - 1)$.

In contrast to quicksort, quickselect makes only one recursive call instead of two. The worst case of quickselect is identical to that of quicksort and is $O(N^2)$. Intuitively, this is because quicksort's worst case is when one of S_1 and S_2 is empty; thus, quickselect is not

really saving a recursive call. The average running time, however, is $O(N)$. The analysis is similar to quicksort's and is left as an exercise.

The implementation of quickselect is even simpler than the abstract description might imply. The code to do this is shown in Figure 7.19. When the algorithm terminates, the

```
/**
 * Internal selection method that makes recursive calls.
 * Uses median-of-three partitioning and a cutoff of 10.
 * Places the kth smallest item in a[k-1].
 * a is an array of Comparable items.
 * left is the left-most index of the subarray.
 * right is the right-most index of the subarray.
 * k is the desired rank (1 is minimum) in the entire array.
 */
template <typename Comparable>
void quickSelect( vector<Comparable> & a, int left, int right, int k )
{
    if( left + 10 <= right )
    {
        const Comparable & pivot = median3( a, left, right );

            // Begin partitioning
        int i = left, j = right - 1;
        for( ; ; )
        {
            while( a[ ++i ] < pivot ) { }
            while( pivot < a[ --j ] ) { }
            if( i < j )
                std::swap( a[ i ], a[ j ] );
            else
                break;
        }

        std::swap( a[ i ], a[ right - 1 ] );  // Restore pivot

            // Recurse; only this part changes
        if( k <= i )
            quickSelect( a, left, i - 1, k );
        else if( k > i + 1 )
            quickSelect( a, i + 1, right, k );
    }
    else  // Do an insertion sort on the subarray
        insertionSort( a, left, right );
}
```

Figure 7.19 Main quickselect routine

kth smallest element is in position $k - 1$ (because arrays start at index 0). This destroys the original ordering; if this is not desirable, then a copy must be made.

Using a median-of-three pivoting strategy makes the chance of the worst case occurring almost negligible. By carefully choosing the pivot, however, we can eliminate the quadratic worst case and ensure an $O(N)$ algorithm. The overhead involved in doing this is considerable, so the resulting algorithm is mostly of theoretical interest. In Chapter 10, we will examine the linear-time worst-case algorithm for selection, and we shall also see an interesting technique of choosing the pivot that results in a somewhat faster selection algorithm in practice.

7.8 A General Lower Bound for Sorting

Although we have $O(N \log N)$ algorithms for sorting, it is not clear that this is as good as we can do. In this section, we prove that any algorithm for sorting that uses only comparisons requires $\Omega(N \log N)$ comparisons (and hence time) in the worst case, so that mergesort and heapsort are optimal to within a constant factor. The proof can be extended to show that $\Omega(N \log N)$ comparisons are required, even on average, for any sorting algorithm that uses only comparisons, which means that quicksort is optimal on average to within a constant factor.

Specifically, we will prove the following result: Any sorting algorithm that uses only comparisons requires $\lceil \log(N!) \rceil$ comparisons in the worst case and $\log(N!)$ comparisons on average. We will assume that all N elements are distinct, since any sorting algorithm must work for this case.

7.8.1 Decision Trees

A **decision tree** is an abstraction used to prove lower bounds. In our context, a decision tree is a binary tree. Each node represents a set of possible orderings, consistent with comparisons that have been made, among the elements. The results of the comparisons are the tree edges.

The decision tree in Figure 7.20 represents an algorithm that sorts the three elements a, b, and c. The initial state of the algorithm is at the root. (We will use the terms *state* and *node* interchangeably.) No comparisons have been done, so all orderings are legal. The first comparison that *this particular* algorithm performs compares a and b. The two results lead to two possible states. If $a < b$, then only three possibilities remain. If the algorithm reaches node 2, then it will compare a and c. Other algorithms might do different things; a different algorithm would have a different decision tree. If $a > c$, the algorithm enters state 5. Since there is only one ordering that is consistent, the algorithm can terminate and report that it has completed the sort. If $a < c$, the algorithm cannot do this, because there are two possible orderings and it cannot possibly be sure which is correct. In this case, the algorithm will require one more comparison.

Every algorithm that sorts by using only comparisons can be represented by a decision tree. Of course, it is only feasible to draw the tree for extremely small input sizes. The number of comparisons used by the sorting algorithm is equal to the depth of the deepest

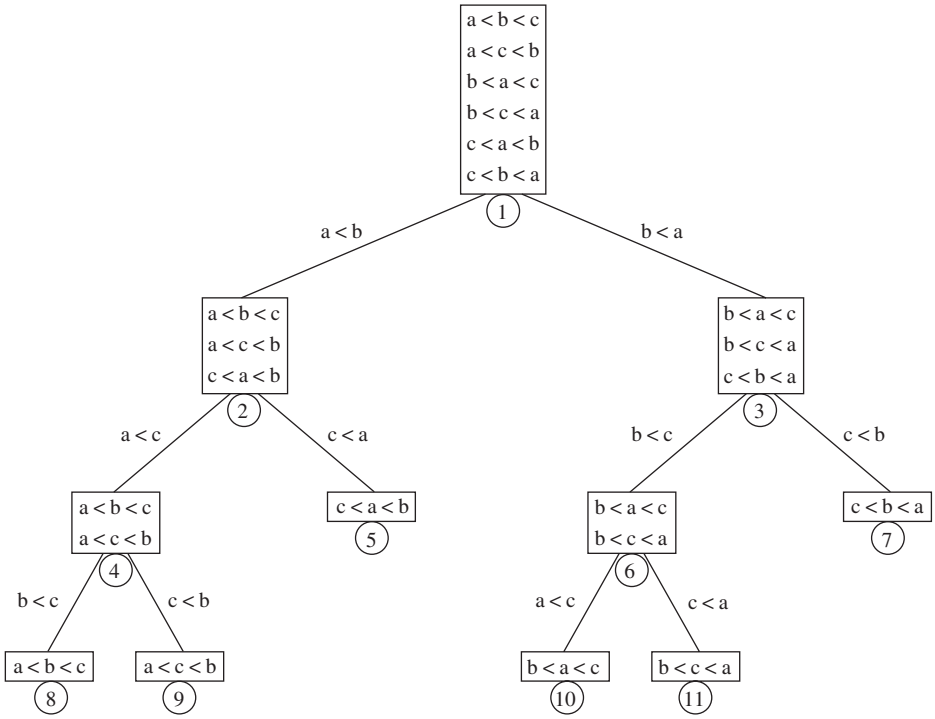

Figure 7.20 A decision tree for three-element sort

leaf. In our case, this algorithm uses three comparisons in the worst case. The average number of comparisons used is equal to the average depth of the leaves. Since a decision tree is large, it follows that there must be some long paths. To prove the lower bounds, all that needs to be shown are some basic tree properties.

Lemma 7.1
Let T be a binary tree of depth d. Then T has at most 2^d leaves.

Proof
The proof is by induction. If $d = 0$, then there is at most one leaf, so the basis is true. Otherwise, we have a root, which cannot be a leaf, and a left and right subtree, each of depth at most $d - 1$. By the induction hypothesis, they can each have at most 2^{d-1} leaves, giving a total of at most 2^d leaves. This proves the lemma.

Lemma 7.2
A binary tree with L leaves must have depth at least $\lceil \log L \rceil$.

Proof
Immediate from the preceding lemma.

Theorem 7.6
Any sorting algorithm that uses only comparisons between elements requires at least $\lceil \log(N!) \rceil$ comparisons in the worst case.

Proof
A decision tree to sort N elements must have $N!$ leaves. The theorem follows from the preceding lemma.

Theorem 7.7
Any sorting algorithm that uses only comparisons between elements requires $\Omega(N \log N)$ comparisons.

Proof
From the previous theorem, $\log(N!)$ comparisons are required.

$$\begin{aligned}
\log(N!) &= \log(N(N-1)(N-2)\cdots(2)(1)) \\
&= \log N + \log(N-1) + \log(N-2) + \cdots + \log 2 + \log 1 \\
&\geq \log N + \log(N-1) + \log(N-2) + \cdots + \log(N/2) \\
&\geq \frac{N}{2} \log \frac{N}{2} \\
&\geq \frac{N}{2} \log N - \frac{N}{2} \\
&= \Omega(N \log N)
\end{aligned}$$

This type of lower-bound argument, when used to prove a worst-case result, is sometimes known as an **information-theoretic** lower bound. The general theorem says that if there are P different possible cases to distinguish, and the questions are of the form YES/NO, then $\lceil \log P \rceil$ questions are always required in some case by any algorithm to solve the problem. It is possible to prove a similar result for the average-case running time of any comparison-based sorting algorithm. This result is implied by the following lemma, which is left as an exercise: Any binary tree with L leaves has an average depth of at least $\log L$.

7.9 Decision-Tree Lower Bounds for Selection Problems

Section 7.8 employed a decision-tree argument to show the fundamental lower bound that any comparison-based sorting algorithm must use roughly $N \log N$ comparisons. In this section, we show additional lower bounds for selection in an N-element collection, specifically

1. $N - 1$ comparisons are necessary to find the smallest item.
2. $N + \lceil \log N \rceil - 2$ comparisons are necessary to find the two smallest items.
3. $\lceil 3N/2 \rceil - O(\log N)$ comparisons are necessary to find the median.

The lower bounds for all these problems, with the exception of finding the median, are tight: Algorithms exist that use exactly the specified number of comparisons. In all our proofs, we assume all items are unique.

Lemma 7.3
If all the leaves in a decision tree are at depth d or higher, the decision tree must have at least 2^d leaves.

Proof
Note that all nonleaf nodes in a decision tree have two children. The proof is by induction and follows Lemma 7.1.

The first lower bound, for finding the smallest item, is the easiest and most trivial to show.

Theorem 7.8
Any comparison-based algorithm to find the smallest element must use at least $N - 1$ comparisons.

Proof
Every element, x, except the smallest element, must be involved in a comparison with some other element, y, in which x is declared larger than y. Otherwise, if there were two different elements that had not been declared larger than any other elements, then either could be the smallest.

Lemma 7.4
The decision tree for finding the smallest of N elements must have at least 2^{N-1} leaves.

Proof
By Theorem 7.8, all leaves in this decision tree are at depth $N - 1$ or higher. Then this lemma follows from Lemma 7.3.

The bound for selection is a little more complicated and requires looking at the structure of the decision tree. It will allow us to prove lower bounds for problems 2 and 3 on our list.

Lemma 7.5
The decision tree for finding the kth smallest of N elements must have at least $\binom{N}{k-1} 2^{N-k}$ leaves.

Proof
Observe that any algorithm that correctly identifies the kth smallest element t must be able to prove that all other elements x are either larger than or smaller than t. Otherwise, it would be giving the same answer regardless of whether x was larger or smaller than t, and the answer cannot be the same in both circumstances. Thus each leaf in the tree, in addition to representing the kth smallest element, also represents the $k - 1$ smallest items that have been identified.

Let T be the decision tree, and consider two sets: $S = \{ x_1, x_2, \ldots, x_{k-1} \}$, representing the $k - 1$ smallest items, and R which are the remaining items, including the

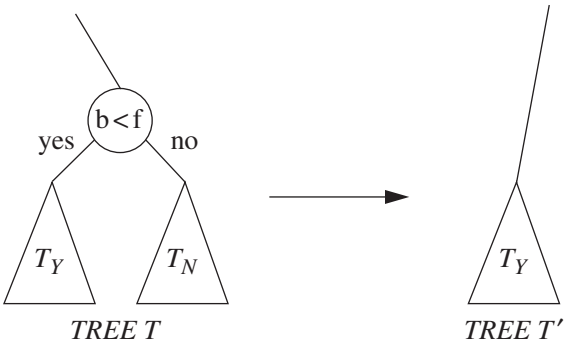

Figure 7.21 Smallest three elements are $S = \{ a, b, c \}$; largest four elements are $R = \{ d, e, f, g \}$; the comparison between b and f for this choice of R and S can be eliminated when forming tree T'

kth smallest. Form a new decision tree, T', by purging any comparisons in T between an element in S and an element in R. Since any element in S is smaller than an element in R, the comparison tree node and its right subtree may be removed from T without any loss of information. Figure 7.21 shows how nodes can be pruned.

Any permutation of R that is fed into T' follows the same path of nodes and leads to the same leaf as a corresponding sequence consisting of a permutation of S followed by the same permutation of R. Since T identifies the overall kth smallest element, and the smallest element in R is that element, it follows that T' identifies the smallest element in R. Thus T' must have at least $2^{|R|-1} = 2^{N-k}$ leaves. These leaves in T' directly correspond to 2^{N-k} leaves representing S. Since there are $\binom{N}{k-1}$ choices for S, there must be at least $\binom{N}{k-1} 2^{N-k}$ leaves in T.

A direct application of Lemma 7.5 allows us to prove the lower bounds for finding the second smallest element and the median.

Theorem 7.9

Any comparison-based algorithm to find the kth smallest element must use at least $N - k + \left\lceil \log \binom{N}{k-1} \right\rceil$ comparisons.

Proof

Immediate from Lemma 7.5 and Lemma 7.2.

Theorem 7.10

Any comparison-based algorithm to find the second smallest element must use at least $N + \lceil \log N \rceil - 2$ comparisons.

Proof

Applying Theorem 7.9, with $k = 2$ yields $N - 2 + \lceil \log N \rceil$.

Theorem 7.11
Any comparison-based algorithm to find the median must use at least $\lceil 3N/2 \rceil - O(\log N)$ comparisons.

Proof
Apply Theorem 7.9, with $k = \lceil N/2 \rceil$.

The lower bound for selection is not tight, nor is it the best known; see the references for details.

7.10 Adversary Lower Bounds

Although decision-tree arguments allowed us to show lower bounds for sorting and some selection problems, generally the bounds that result are not that tight, and sometimes are trivial.

For instance, consider the problem of finding the minimum item. Since there are N possible choices for the minimum, the information theory lower bound that is produced by a decision-tree argument is only $\log N$. In Theorem 7.8, we were able to show the $N - 1$ bound by using what is essentially an **adversary argument**. In this section, we expand on this argument and use it to prove the following lower bound:

4. $\lceil 3N/2 \rceil - 2$ comparisons are necessary to find both the smallest and largest item

Recall our proof that any algorithm to find the smallest item requires at least $N - 1$ comparisons:

Every element, x, except the smallest element, must be involved in a comparison with some other element, y, in which x is declared larger than y. Otherwise, if there were two different elements that had not been declared larger than any other elements, then either could be the smallest.

This is the underlying idea of an adversary argument which has some basic steps:

1. Establish that some basic amount of information must be obtained by any algorithm that solves a problem.
2. In each step of the algorithm, the adversary will maintain an input that is consistent with all the answers that have been provided by the algorithm thus far.
3. Argue that with insufficient steps, there are multiple consistent inputs that would provide different answers to the algorithm; hence, the algorithm has not done enough steps, because if the algorithm were to provide an answer at that point, the adversary would be able to show an input for which the answer is wrong.

To see how this works, we will re-prove the lower bound for finding the smallest element using this proof template.

Theorem 7.8 *(restated)*
Any comparison-based algorithm to find the smallest element must use at least $N - 1$ comparisons.

New Proof

Begin by marking each item as U (for unknown). When an item is declared larger than another item, we will change its marking to E (for eliminated). This change represents one unit of information. Initially each unknown item has a value of 0, but there have been no comparisons, so this ordering is consistent with prior answers.

A comparison between two items is either between two unknowns or it involves at least one item eliminated from being the minimum. Figure 7.22 shows how our adversary will construct the input values, based on the questioning.

If the comparison is between two unknowns, the first is declared the smaller and the second is automatically eliminated, providing one unit of information. We then assign it (irrevocably) a number larger than 0; the most convenient is the number of eliminated items. If a comparison is between an eliminated number and an unknown, the eliminated number (which is larger than 0 by the prior sentence) will be declared larger, and there will be no changes, no eliminations, and no information obtained. If two eliminated numbers are compared, then they will be different, and a consistent answer can be provided, again with no changes, and no information provided.

At the end, we need to obtain $N-1$ units of information, and each comparison provides only 1 unit at the most; hence, at least $N - 1$ comparisons are necessary.

Lower Bound for Finding the Minimum and Maximum

We can use this same technique to establish a lower bound for finding both the minimum and maximum item. Observe that all but one item must be eliminated from being the smallest, and all but one item must be eliminated from being the largest; thus the total information that any algorithm must acquire is $2N - 2$. However, a comparison $x < y$ eliminates both x from being the maximum and y from being the minimum; thus, a comparison can provide two units of information. Consequently, this argument yields only the trivial $N - 1$ lower bound. Our adversary needs to do more work to ensure that it does not give out two units of information more than it needs to.

To achieve this, each item will initially be unmarked. If it "wins" a comparison (i.e., it is declared larger than some item), it obtains a W. If it "loses" a comparison (i.e., it is declared smaller than some item), it obtains an L. At the end, all but two items will be WL. Our adversary will ensure that it only hands out two units of information if it is comparing two unmarked items. That can happen only $\lfloor N/2 \rfloor$ times; then the remaining information has to be obtained one unit at a time, which will establish the bound.

x	y	Answer	Information	New x	New y
U	U	$x < y$	1	No change	Mark y as E Change y to #elim
All others		Consistently	0	No change	No change

Figure 7.22 Adversary constructs input for finding the minimum as algorithm runs

Theorem 7.12
Any comparison-based algorithm to find the minimum and maximum must use at least $\lceil 3N/2 \rceil - 2$ comparisons.

Proof
The basic idea is that if two items are unmarked, the adversary must give out two pieces of information. Otherwise, one of the items has either a W or an L (perhaps both). In that case, with reasonable care, the adversary should be able to avoid giving out two units of information. For instance, if one item, x, has a W and the other item, y, is unmarked, the adversary lets x win again by saying $x > y$. This gives one unit of information for y but no new information for x. It is easy to see that, in principle, there is no reason that the adversary should have to give more than one unit of information out if there is at least one unmarked item involved in the comparison.

It remains to show that the adversary can maintain values that are consistent with its answers. If both items are unmarked, then obviously they can be safely assigned values consistent with the comparison answer; this case yields two units of information.

Otherwise, if one of the items involved in a comparison is unmarked, it can be assigned a value the first time, consistent with the other item in the comparison. This case yields one unit of information.

x	y	Answer	Information	New x	New y
–	–	$x < y$	2	L 0	W 1
L	–	$x < y$	1	L unchanged	W $x + 1$
W or WL	–	$x > y$	1	W or WL unchanged	L $x - 1$
W or WL	W	$x < y$	1 or 0	WL unchanged	W $max(x + 1, y)$
L or W or WL	L	$x > y$	1 or 0 or 0	WL or W or WL unchanged	L $min(x - 1, y)$
WL	WL	consistent	0	unchanged	unchanged
– – – L L W	W WL L W WL WL	SYMMETRIC TO AN ABOVE CASE			

Figure 7.23 Adversary constructs input for finding the maximum and minimum as algorithm runs

Otherwise, both items involved in the comparison are marked. If both are WL, then we can answer consistently with the current assignment, yielding no information.[1]

Otherwise, at least one of the items has only an L or only a W. We will allow that item to compare redundantly (if it is an L then it loses again; if it is a W then it wins again), and its value can be easily adjusted if needed, based on the other item in the comparison (an L can be lowered as needed; a W can be raised as needed). This yields at most one unit of information for the other item in the comparison, possibly zero. Figure 7.23 summarizes the action of the adversary, making y the primary element whose value changes in all cases.

At most $\lfloor N/2 \rfloor$ comparisons yield two units of information, meaning that the remaining information, namely $2N - 2 - 2\lfloor N/2 \rfloor$ units, must each be obtained one comparison at a time. Thus the total number of comparisons that are needed is at least $2N - 2 - \lfloor N/2 \rfloor = \lceil 3N/2 \rceil - 2$.

It is easy to see that this bound is achievable. Pair up the elements, and perform a comparison between each pair. Then find the maximum among the winners and the minimum among the losers.

7.11 Linear-Time Sorts: Bucket Sort and Radix Sort

Although we proved in Section 7.8 that any general sorting algorithm that uses only comparisons requires $\Omega(N \log N)$ time in the worst case, recall that it is still possible to sort in linear time in some special cases.

A simple example is **bucket sort**. For bucket sort to work, extra information must be available. The input $A_1, A_2, \ldots, A_N$ must consist of only positive integers smaller than M. (Obviously extensions to this are possible.) If this is the case, then the algorithm is simple: Keep an array called `count`, of size M, which is initialized to all 0s. Thus, `count` has M cells, or buckets, which are initially empty. When A_i is read, increment `count[`A_i`]` by 1. After all the input is read, scan the `count` array, printing out a representation of the sorted list. This algorithm takes $O(M + N)$; the proof is left as an exercise. If M is $O(N)$, then the total is $O(N)$.

Although this algorithm seems to violate the lower bound, it turns out that it does not because it uses a more powerful operation than simple comparisons. By incrementing the appropriate bucket, the algorithm essentially performs an M-way comparison in unit time. This is similar to the strategy used in extendible hashing (Section 5.9). This is clearly not in the model for which the lower bound was proven.

This algorithm does, however, question the validity of the model used in proving the lower bound. The model actually is a strong model, because a *general-purpose* sorting algorithm cannot make assumptions about the type of input it can expect to see but must

[1] It is possible that the current assignment for both items has the same number; in such a case we can increase all items whose current value is larger than y by 2, and then add 1 to y to break the tie.